

1)

I set the random seed to 432 and split the data into 70% training and 30% test observations. I standardized the three predictors using the training means and standard deviations, then applied the same transformation to the test data. This keeps information from the test set out of the model fitting process.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import roc_curve, roc_auc_score

# Load the data and check the variables used in the analysis.
data = pd.read_csv("breast-cancer.csv")
predictors = ["mean_radius", "mean_texture", "mean_smoothness"]
X = data[predictors]
y = data["y"]
assert data.shape[0] == 569
assert not data[predictors + ["y"]].isna().any().any()
assert set(y.unique()) == {0, 1}

# Keep this split and scaling for Question 2 as well.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=432
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print(f"Training observations: {len(y_train)}")
print(f"Test observations: {len(y_test)}")

models = {
    "Logistic regression": LogisticRegression(
        C=np.inf, fit_intercept=True, solver="lbfgs",
        max_iter=10000, tol=1e-10
    )
}
for k in [5, 15, 25, 35, 45]:
    models[f"KNN (k={k})"] = KNeighborsClassifier(
        n_neighbors=k, weights="uniform", metric="euclidean"
    )

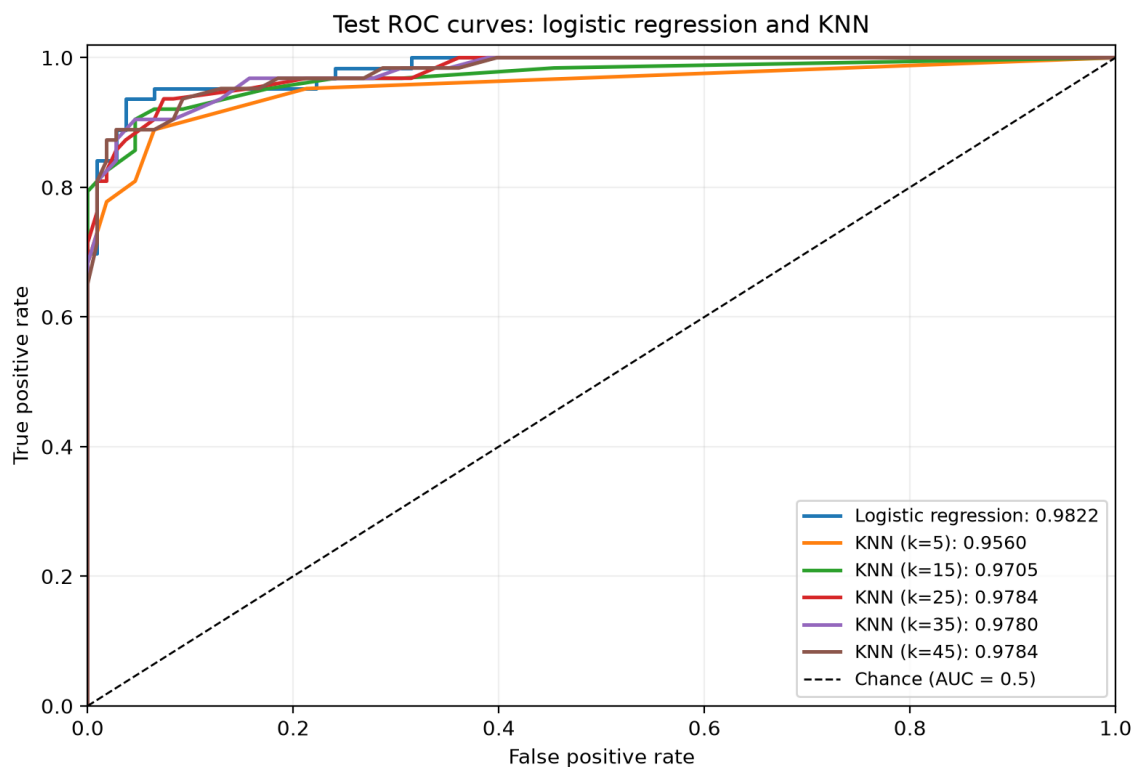
fig, ax = plt.subplots(figsize=(8, 5.5))
rows = []
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    malignant_column = list(model.classes_).index(1)
    probabilities = model.predict_proba(X_test_scaled)[: , malignant_column]
    auc = roc_auc_score(y_test, probabilities)
    fpr, tpr, _ = roc_curve(y_test, probabilities)
    rows.append({"Model": name, "Test AUC": auc})
    ax.plot(fpr, tpr, linewidth=1.8, label=f"{name}: {auc:.4f}")

ax.plot([0, 1], [0, 1], "k--", linewidth=1, label="Chance (AUC = 0.5)")
ax.set(xlabel="False positive rate", ylabel="True positive rate",
       title="Test ROC curves: logistic regression and KNN",
       xlim=(0, 1), ylim=(0, 1.02))
ax.legend(loc="lower right", fontsize=9)
ax.grid(alpha=0.2)
fig.tight_layout()
fig.savefig("homework-06-q1-roc.png", dpi=200, bbox_inches="tight")
plt.show()

auc_table = pd.DataFrame(rows)
print(auc_table.to_string(index=False, float_format=lambda x: f"{x:.4f}"))
```

The split gave me 398 training observations and 171 test observations. There were no missing values in the variables used for this analysis. I fit logistic regression with an intercept and no penalty, and used Euclidean distance and equal neighbor weights for each KNN model. For the ROC curves, I used each model's predicted probability of a malignant case ($y = 1$).

Model	Test AUC
Logistic regression	0.9822
KNN, $k = 5$	0.9560
KNN, $k = 15$	0.9705
KNN, $k = 25$	0.9784
KNN, $k = 35$	0.9780
KNN, $k = 45$	0.9784



All six models performed well on the test set, with AUC values above 0.95. I would choose logistic regression because it had the highest test AUC, at 0.9822. This means it was the best of these models at ranking malignant cases above benign cases across classification thresholds. Its AUC was only slightly higher than those for KNN with $k = 25, 35$, and 45 , so the difference was small.

Among the KNN models, $k = 5$ had the lowest AUC. Performance improved with larger neighborhoods and was very similar for $k = 25, 35$, and 45 . These results support choosing logistic regression for this split, although a different random split could change the ranking.

2)

I used the same 398 training observations and 171 test observations from Question 1, with the same standardization. For each value of k , I fit one KNN model and used it to predict probabilities for both datasets. Each training observation was included among its own nearest neighbors, as required.

```
# Reuse the training/test split and standardized predictors from Question 1.
k_values = [1, 3, 5, 10, 20]
fig2, axes = plt.subplots(2, 3, figsize=(11, 7), sharex=True, sharey=True)
q2_rows = []

for ax, k in zip(axes.flat, k_values):
    knn = KNeighborsClassifier(
        n_neighbors=k, weights="uniform", metric="euclidean"
    )
    knn.fit(X_train_scaled, y_train)
    malignant_column = list(knn.classes_).index(1)

    # Passing the training observations explicitly includes each point itself.
    train_prob = knn.predict_proba(X_train_scaled)[:, malignant_column]
    test_prob = knn.predict_proba(X_test_scaled)[:, malignant_column]
    train_auc = roc_auc_score(y_train, train_prob)
    test_auc = roc_auc_score(y_test, test_prob)
    q2_rows.append({"k": k, "Training AUC": train_auc, "Test AUC": test_auc})

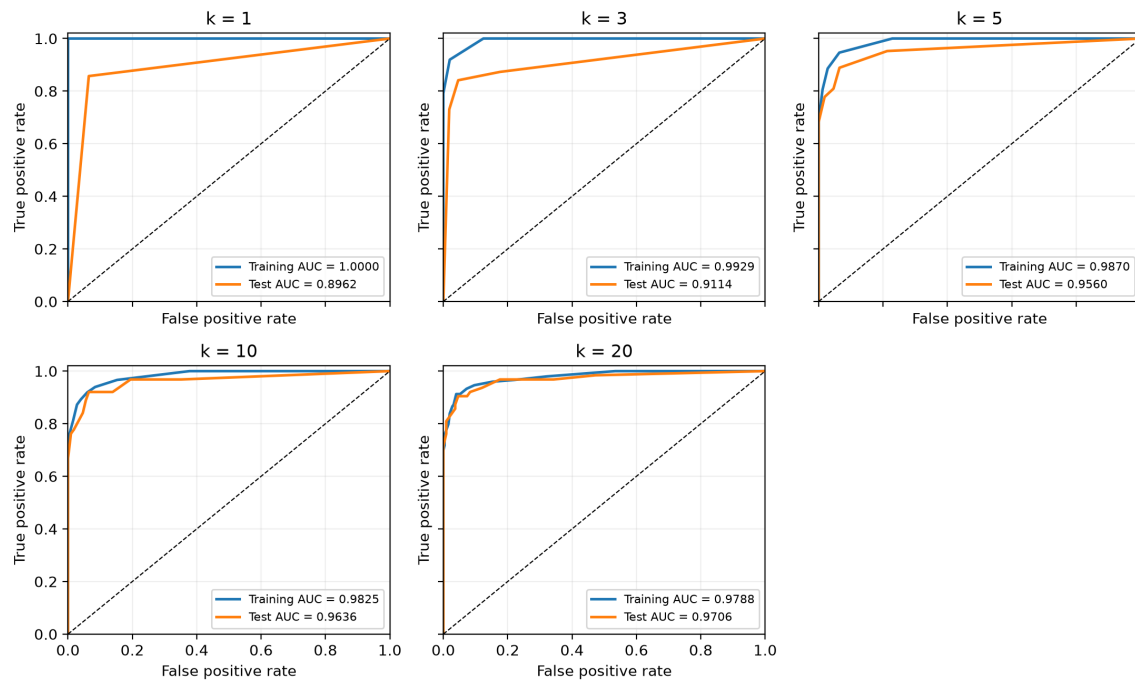
    for labels, prob, name, auc, color in [
        (y_train, train_prob, "Training", train_auc, "tab:blue"),
        (y_test, test_prob, "Test", test_auc, "tab:orange")
    ]:
        fpr, tpr, _ = roc_curve(labels, prob)
        ax.plot(fpr, tpr, color=color, linewidth=1.8,
                label=f"{name} AUC = {auc:.4f}")
        ax.plot([0, 1], [0, 1], "k--", linewidth=0.8)
        ax.set(title=f"k = {k}", xlim=(0, 1), ylim=(0, 1.02))
        ax.set_xlabel("False positive rate")
        ax.set_ylabel("True positive rate")
        ax.legend(loc="lower right", fontsize=8)
        ax.grid(alpha=0.2)

axes.flat[-1].set_visible(False)
fig2.suptitle("KNN: training and test ROC curves", fontsize=14)
fig2.tight_layout()
fig2.savefig("homework-06-q2-roc.png", dpi=200, bbox_inches="tight")
plt.show()

q2_auc_table = pd.DataFrame(q2_rows)
print(q2_auc_table.to_string(index=False, float_format=lambda x: f"{x:.4f}"))
```

k	Training AUC	Test AUC
1	1.0000	0.8962
3	0.9929	0.9114
5	0.9870	0.9560
10	0.9825	0.9636
20	0.9788	0.9706

KNN: training and test ROC curves



The training AUC was higher than the test AUC for every value of k . As k increased from 1 to 20, the training AUC decreased slightly, while the test AUC increased. The gap narrowed from about 0.104 at $k = 1$ to about 0.008 at $k = 20$. The ROC panels show the same pattern: the training and test curves became closer overall as the neighborhood got larger.

At $k = 1$, the training AUC was exactly 1. Each training observation was its own nearest neighbor, so its predicted probability was 1 if it was malignant and 0 if it was benign. This gave perfect separation of the training labels. However, the test AUC was only 0.8962 because a new observation had to use the label of its nearest training observation, which was not always the same as its own label. Since the probabilities at $k = 1$ can only be 0 or 1, its ROC curve also has fewer distinct threshold points than the curves for larger k .

This shows why perfect training performance does not necessarily mean a model will perform well on new data. With a small k , KNN is very sensitive to individual training observations and can overfit. Increasing k averages over more neighbors, making the predictions less sensitive to individual points. In this split, that improved test performance even though training performance decreased. Of these five choices, $k = 20$ had the highest test AUC, at 0.9706. This does not mean that increasing k will always help; a neighborhood that is too large can smooth over useful differences between the two classes.

3)

I reviewed [hethp's Homework 5 report](#), Question 1(b).

One thing that could be improved is that the R code uses `f0` when calculating bias and MSE, but `f0` is never defined in the code shown. For example, the report includes:

```
bias <- colMeans(pred) - f0
```

Even though the value of `f(x_0)` is calculated earlier in the written explanation, the code should also include a line defining `f0`. Without that line, the code would give an error if someone tried to run it as shown in a fresh R session.

To fix this, add the following line before calculating bias and MSE:

```
f0 <- 0.5*0.5 + sin(0.7) - 0.3*1^2
```

This gives approximately 0.5942. Adding the definition fixes the undefined-variable error and makes this part of the code reproducible. After adding it, rerun the Question 1 R code in a fresh session to check that `f0` is available for both calculations.

Great work otherwise!

Posted GitHub Issue: [Homework 5, Question 1\(b\): Define f0 before calculating bias and MSE](#)